Raf Library Management Platform Internship Report

Student: Oğuzhan Yıldırtan | Student number: 123200124 | Course: CMPE 200

Institution: Vadi Kurumsal Bilgi Sistemleri | Supervisor: Mustafa Arı

Internship period: 27 July 2026 to 21 August 2026

Question 1 Information about the institution

Name and address

I completed my internship at Vadi Kurumsal Bilgi Sistemleri, a software company serving public institutions and local authorities. My supervisor was Mustafa Arı. The project described in this report is the Raf Library Management Platform.

Address: Çifte Havuzlar, YTÜ Davutpaşa Kampüsü, Yıldız Teknik Üniversitesi, A1 Blok No:207, 34220 Esenler/İstanbul.

Brief history

Founded in 1998, the company develops web-based software for e-government and municipal services. Its services include software development, system integration and project consultancy. It uses object-oriented methods and layered architectures to support maintenance and scaling.

Parent organisation

The company profile provided for this report does not specify a parent organisation.

Existing facilities

The company's headquarters are in Istanbul, at Yıldız Technical University's Davutpaşa Campus. Its team works in software development, technical support and administration.

Number of employees

During my internship, approximately 20 employees worked at the office where I was based.

Question 2 Hardware and software used

The development hardware was my own MacBook Air with an Apple M3 processor, running macOS. I used a terminal and a web browser, together with a local PostgreSQL server. The

browser was used to inspect the application, while terminal commands started the development services and ran builds and tests.

Frontend and backend technologies

React and TypeScript supported the interface and data types. Vite provided development and production builds, CSS defined the layout, and lucide-react supplied icons.

NestJS organised API controllers and services. The class-validator and class-transformer libraries checked input, and the PostgreSQL driver supported parameterised queries, pooling and transactions.

The Model Context Protocol SDK and Zod defined the assistant's tools. The OpenAI JavaScript SDK connected to Gemini's compatible endpoint. Vitest supported unit tests, pnpm managed the workspace, and Git supported version control with a .gitignore file.

Question 3 Assigned work and project participation

My project work focused on a library management application for staff. The application brings member registration, catalogue browsing, lending, returns and overdue monitoring into one interface. A separate assistant allows questions about the catalogue and active loan records through predefined tools. The application was developed as a learning project using demonstration data.

The technical responsibilities covered the user interface, API endpoints, database queries, circulation rules and integration of the assistant. My internship supervisor was Mustafa Arı.

System architecture

The workspace contains three applications: a React web frontend, a NestJS API and an MCP server. The frontend sends HTTP requests to the API, which queries PostgreSQL. For assistant requests, the API checks the question's scope, contacts Gemini and forwards requested tool calls to the MCP server over standard input and output. The MCP server executes predefined, read-only SQL queries and returns their results.

The frontend handles presentation, the API applies business rules, and PostgreSQL stores the records. Catalogue and circulation operations can remain available without a configured Gemini key.

The member module records a name, a Turkish identity number, a birth date and optional contact details. It validates the identity number's format and checksum and masks the number in list responses. This algorithm does not verify a person's identity against an official registry.

The catalogue shows each copy's title, author, category, shelf and availability. A copy can be added with a unique inventory code and optional ISBN. The service reuses matching bibliographic records where possible and creates each copy within a transaction.

Physical copies and circulation

The model was revised to separate bibliographic information in books from physical items in book_copies. Each copy has its own inventory code and location, and loans refer to that copy. This allows two copies of the same title to be lent independently.

Legacy copy records preserve the associations of existing loans. Lending checks the selected copy and member, locks the record and creates a loan within a transaction. A partial unique index prevents two active loans for the same copy.

Returning a copy records a return date and restores its availability. Active loan lists display borrowing and due dates, while overdue queries compare due dates with the database's current date.

The dashboard summarises physical copies, members, active loans and available copies. It also lists overdue returns and links to common tasks. The sidebar connects the dashboard, members, circulation, catalogue and assistant screens.

The interface includes registration forms, catalogue search and an availability filter. Data refreshes after successful operations. Visible loading errors distinguish failed requests from empty lists, and the welcome area's date now updates automatically.

Assistant integration

Five MCP tools support book search, book status, member loans, overdue loans and available books. Their names are search_books, get_book_status, get_member_loans, list_overdue_loans and list_available_books. Each executes a predefined query; the model has no free-form SQL tool.

A scope filter checks library-related, sensitive and unrelated questions. Identity numbers, birth dates and contact details are excluded from tool results. Member names and loan information can still appear, so the results are not anonymous.

The assistant reads records, while registration, lending and returns remain staff-interface operations. A limit on model and tool iterations prevents an indefinite loop. Generated answers require a valid Gemini configuration and provider availability.

Question 4 Problems encountered and solutions

The original title-level constraint prevented multiple copies of one book from being represented. Introducing book_copies and copy-specific loans addressed this problem and allowed independent availability tracking.

Concurrent requests could otherwise lend the same copy twice. The loan service combines a transaction and row lock with a database index allowing only one active loan per copy.

The member-name uniqueness constraint was removed because different people can share a name. The identity-number uniqueness rule was retained to detect duplicate member records.

Text transformation now removes surrounding spaces before validation. This prevents a name consisting only of spaces from satisfying a length check and aligns validation with the value stored.

The form initially allowed today's date although the service required a later due date. The form's minimum was changed to tomorrow, the service uses Europe/Istanbul for today's date, and frontend date generation uses local calendar values.

The book-status query depended on PostgreSQL's similarity function and the pg_trgm extension. Ordinary ordering replaced that dependency, avoiding a failed query followed by a fallback on installations without the extension.

Tests were included outside the API's declared source root, causing a standalone type check to fail. A separate test configuration now checks tests without emitting output.

Verification and remaining limitations

The 12 unit tests in two files passed. They cover identity-number validation and masking, plus the chat scope filter's library, sensitive-data and follow-up cases. Application builds were also checked during development.

The tests do not cover complete lending workflows, migrations or live Gemini calls. Database integration tests would strengthen verification. The text-based scope filter also remains a prototype-level control rather than a complete security boundary.

Question 5 Professional skills and learning outcomes

The project helped me trace a user action from a React form through API validation to a database update. I learned why frontend behaviour, backend checks and database constraints must represent the same business rule.

Separating books from physical copies improved my understanding of relational modelling. Transactions, row locks and unique indexes showed how to protect lending operations. Removing name uniqueness demonstrated the importance of matching constraints to real-world data.

Validation and date corrections improved my debugging skills. I examined input transformation, service checks and storage types together. Build configuration work clarified the difference between compiling application code and checking tests.

The assistant integration taught me how structured model tool calls connect generated answers to database records. It also helped me describe the limits of sensitive-field restrictions and automated tests accurately.

Question 6 General evaluation of the internship

The internship project allowed me to apply web development and database concepts in a working prototype. Its main result was a staff interface with member records, copy-based circulation and an optional assistant. Supporting multiple copies showed how changing requirements can affect the database and application together.

Tracing errors across interface behaviour, service logic and SQL was especially useful. Reviewing these layers together made the application more consistent and helped me understand the reasons behind each correction.

The project remains a demonstration. Future work could add integration tests, controlled migrations and better handling of large catalogues. Authentication and additional assistant testing would be needed before real organisational use.

The project provided practical experience in designing, implementing and reviewing an application. My next learning objective is to verify complete workflows with repeatable integration tests and improve the installation documentation.